A Practical Guide to Multi-Objective Reinforcement Learning and Planning

Parisa Babolhavaeji, Ryan C Bowers, Bhavini Jeloka, Ghazal Mirzazadeh, Karthic Palaniappan

1 Introduction

The authors of Hayes et al. [2021] present a comprehensive overview of current research in multi-objective reinforcement learning (MORL) and planning, encompassing fundamental definitions, underlying motivations, and practical implementations. MORL addresses the complex challenge of simultaneously optimizing multiple, often conflicting objectives, a scenario common across a broad spectrum of decision-making problems—from small-scale tasks such as travel planning to large-scale applications including defense procurement and wind farm control. For a detailed exposition and illustrative examples, readers are referred to Hayes et al. [2021].

Traditional reinforcement learning (RL) methods typically combine all objectives into a single scalar reward function. While this scalarization simplifies optimization, it often compromises the explainability of the decision-making process and necessitates extensive tuning to identify a reward formulation that accurately reflects user preferences. Moreover, such approaches fail to generalize in settings where user preferences are non-stationary. To address these limitations, it is essential to represent each objective through its own reward component and introduce a utility function that explicitly captures user preferences and trade-offs among competing objectives.

The formal framework for multi-objective sequential decision-making is the Multi-Objective Markov Decision Process (MOMDP), defined by the tuple $\langle S, A, T, \gamma, \mu, \mathbf{R}\rangle$. Here, S denotes the state space, A the action space, $T : S \times A \times S \rightarrow [0, 1]$ the probabilistic transition function, $\gamma \in [0, 1)$ the discount factor, and $\mu : S \rightarrow [0, 1]$ the initial state distribution. The key distinction from standard MDPs lies in the reward function $\mathbf{R} : S \times A \times S \rightarrow \mathbb{R}^d$, which returns a d-dimensional vector where each component corresponds to a distinct objective.

The vector-valued reward function $\mathbf{R}$ is what differentiates the MOMDP from the traditional MDP and each element in this vector corresponds to each of the considered objectives. Consequently, the length of the reward vector is equal to the number of objectives.

We additionally define a policy as a mapping $\pi : S \times A \rightarrow [0, 1]$, i.e., for any given state, an action is selected according to a certain probability distribution. The value function of a policy π in a MOMDP is defined as:

$$\mathbf{V}^{\pi}(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k \mathbf{r}_{k+1} \mid \pi, s_o = s\right], \tag{1}$$

where $\mathbf{r}_{k+1} = \mathbf{R}(s_k, a_k, s_{k+1})$ is the reward received at timestep $k + 1$. Notice that the value function is also vector-valued, $\mathbf{V}^{\pi} \in \mathbb{R}^d$. The *utility function* $u : \mathbb{R}^d \rightarrow \mathbb{R}$ maps the aforementioned multi-objective value of a policy to a scalar value,

$$V_u^{\pi} = u(\mathbf{V}^{\pi}) \tag{2}$$

and reduces the MOMDP to a single-objective decision making problem.

Unlike single-objective MDPs where the optimal policy π^* is equivalent to optimizing for the *unique* optimal value function, for an MOMDP this is not necessarily the case. Without knowing the user's utility, there can now be multiple possibly optimal value vectors $\mathbf{V}^{\pi}$. Thus, the problem becomes one of finding optimal *solution sets*. Examples of such solution sets include the undominated set, the coverage set, and the Pareto front. A key challenge in utilizing these sets lies in constraining their size to ensure both tractable learning and efficient deployment. The design of multi-objective systems is further influenced by factors such as the optimization criterion and the solution approach, both of which must be selected with careful consideration.

A commonly used optimization criterion in MORL and planning is the scalarized expected return (SER), defined as:

$$V_u^\pi = u\left(\mathbb{E}\left[\sum_{i=0}^{\infty}\gamma^t \mathbf{r}_i \,\middle|\, \pi, s_0\right]\right). \tag{3}$$

The SER criterion is particularly suitable when the decision-maker's objective is to achieve an optimal long-term utility. For SER, a coverage set is defined as the collection of policies that are optimal for all possible utility functions. In contrast, the expected scalarized return (ESR) formulation reverses the order of scalarization and expectation, reflecting a different assumption about how utility is aggregated over time.

Utility-based approaches help alleviate issues of large solution-set cardinality and sample inefficiency by deriving the optimal solution set from the available knowledge about the utility function of the user, and which types of policies are allowed.

This work builds upon the survey by Hayes et al. Hayes et al. [2021] in two phases. The first phase reproduces the experiments from Hayes et al., implementing MONES on the Dam environment and comparing it against an outer-loop baseline using single-objective NES with linear scalarization. This reproduction validates our experimental setup and establishes baseline performance metrics including hypervolume and expected utility maximization (EUM). The second phase extends beyond reproduction by introducing Multi-Objective Deep Q-Networks (MO-DQN), a value-based alternative to the policy-based MONES algorithm. MO-DQN learns separate Q-functions for each objective and enables deployment-time adaptation to arbitrary linear utility functions without retraining. We evaluate MO-DQN on the original Dam environment to enable direct comparison with MONES, then assess its scalability on three additional benchmark environments from the mo-gymnasium library: Resource Gathering, Multi-Objective Mountain Car, and Deep Sea Treasure. These experiments investigate whether value-based deep RL methods can provide competitive performance and offer improved sample efficiency and deployment flexibility.

2 Modifications

The initial reproduction work implemented Multi-Objective Natural Evolution Strategies (MONES), the algorithm referenced by Hayes et al. [2021]. To the initial implementation of MONES, we introduced Multi-Objective Deep Q-Networks (MO-DQN), a value-based approach that fundamentally differs from the policy-based MONES algorithm. MO-DQN extends the standard DQN architecture to handle vector-valued rewards and, at the same time, maintains compatibility with arbitrary linear utility functions at deployment time. It is more efficient, has a more flexible deployment, and scales better. Table 1 summarizes the key differences between MONES and MO-DQN.

Table 1: Comparison of MONES and MO-DQN approaches

Aspect	**MONES**	**MO-DQN**
Paradigm	Policy-based, Evolutionary	Value-based, Temporal Difference
Learning	Population sampling + natural gradients	Experience replay + target networks
Network Output	Direct action (policy network)	Vector Q-values per objective
Utility Handling	Implicit (Pareto optimization)	Explicit (scalarization at decision time)
Sample Reuse	None (on-policy)	High (off-policy with replay)
Deployment Output	Multiple policies (Pareto set)	Single policy (adapts to any $\mathbf{w}$)

MO-DQN learns a separate Q-function for each objective. Given d objectives, the network outputs $\mathbf{Q}(s,a) \in \mathbb{R}^d$, where each component $Q_i(s,a)$ represents the expected discounted return for objective i when taking action a in state s. The architecture consists of a vector Q-network that outputs $|A| \times d$ values representing Q-values for all action-objective pairs. This netwrok works alongside with a target network that is periodically updated to provide stable temporal difference learning targets. Transitions $(s, a, \mathbf{r}, s')$ with vector rewards $\mathbf{r} \in \mathbb{R}^d$ are stored in an experience replay buffer to make off-policy learning possible. At decision time, actions are selected through utility-weighted maximization: $a^* = \arg\max_a \left[\mathbf{w}^\top \mathbf{Q}(s,a)\right]$ for a given utility weight vector $\mathbf{w}$.

The learning process follows standard DQN principles adapted for vector rewards. For each objective i, the loss is computed independently:

$$\mathcal{L}_i = \mathbb{E}\left[\left(r_i + \gamma \max_{a'} Q_i(s', a') - Q_i(s,a)\right)^2\right] \tag{4}$$

The total loss is the sum across objectives $\mathcal{L} = \sum_{i=1}^{d} \mathcal{L}_i$, and gradient descent updates all Q-functions simultaneously. Crucially, the utility weights $\mathbf{w}$ are not used during training and are only applied at decision time. This makes the

deployment more flexible.

As an evolutionary algorithm, Mones samples a population of policies each iteration and evaluates them through complete environment rollouts. Each collected transition is used once for fitness evaluation and then discarded. In contrast, MO-DQN uses experience replay. This allows each transition to be reused multiple times across training updates. This off-policy learning paradigm is particularly valuable in environments where sample collection is expensive or interaction is limited. Decoupling data collection from policy updates makes MO-DQN able to extract more learning signal from the same amount of environment interaction.

A significant advantage of MO-DQN's architecture is that utility weights are decoupled from training. Since the network learns separate Q-functions for each objective, we could apply any linear utility function $u(\mathbf{V}) = \mathbf{w}^\top \mathbf{V}$ at deployment without retraining. This made us able to explore trade-offs without having to recompute. MONES, on the other hand, produces a fixed set of policies when training completes. If we wanted to adapt a new utility function, our options where limited to selecting the nearest existing policy or retraining.

3 Experimental Setup

We implemented our algorithm in 4 multi-objective RL environments (with discrete and continuous state-action spaces, and multi-dimensional rewards) from the MO-Gymnasium library. Figure 1 displays visuals, and Table 2 details properties of all environments. The number and type of actions, dimensions of state and reward spaces, randomness in environment dynamics (Gaussian-based inflow in water-reservoir, Bernoulli-based death by dragon in resource-gathering, etc.) significantly vary across these environments. More details can be found in Felten et al. [2023].

Figure 1: Visuals for water-reservoir, resource-gathering, deep-sea-treasure, mo-mountain-car respectively.

#	Environment	Action-space	State (obs)-dim	Reward-dim
1	deep-sea-treasure-v0	Discrete(4)	2	2
2	resource-gathering-v0	Discrete(4)	4	3
3	mo-mountaincar-v0	Discrete(3)	2	2
4	water-reservoir-v0	Continuous(1)	10	2

Table 2: Properties of multi-objective RL environments used.

We also benchmarked the performance of the multi-objective natural evolution strategies (MONES) algorithms from the original paper, in the water-reservoir-v0 environment, with two objectives (flooding, water demand), one state representing water left in reservoir (floating point), and one action representing water released.

4 Results

In this section, we evaluate MODQN across environments, and compare performance against MONES algorithms.

4.1 Performance of MODQN across environments

We trained MODQN algorithm across 3 environments (resource-gathering-v0, mo-mountaincar-v0, and deep-sea-treasure-v0) for a given utility function. The utility function used here is a linear combination of weights used to derive the objective from a multi-dimensional reward vector. Details of the hyperparameters of the trained model are mentioned in Table 3. The convergence rate varies based on the complexity of the environment, as seen in the figures that follow.

Hyperparameter	resource-gathering-v0	mo-mountaincar-v0	deep-sea-treasure-v0	water-reservoir-v0
num_episodes	10000	20000	20000	500
ep_len	100	100	100	100
warmup_ep_len	100	200	200	100
lr	0.0005	0.0005	0.0005	0.001
gamma	0.99	0.99	0.99	0.99
batch_size	256	256	256	64
utility_fn	[0.4, 0.3, 0.3]	[0.4, 0.4, 0.2]	[0.5, 0.5]	[0.5, 0.5]
layer_sizes	[128, 128]	[64, 64]	[128, 128]	[64, 64]
epsilon_decay	0.9997	0.9997	0.9997	0.999
updates_per_episode	50	50	100	1
max_buffer_size	50000	50000	50000	50000

Table 3: Fine-tuned hyperparameters (grid sweep) of MODQN algorithm for the four environments

Figure 2 shows the training results for resource-gathering-v0 environment. The objective converges to a value of 0.5 (as seen in the first plot), with sub-objectives 1 (returned with gold) and 2 (returned with diamond) converging close to 1. These results indicate the ability of our model to be modified at inference to optimize to either objectives. The gradient in the average loss also indicates the potential to improve stability upon further training.

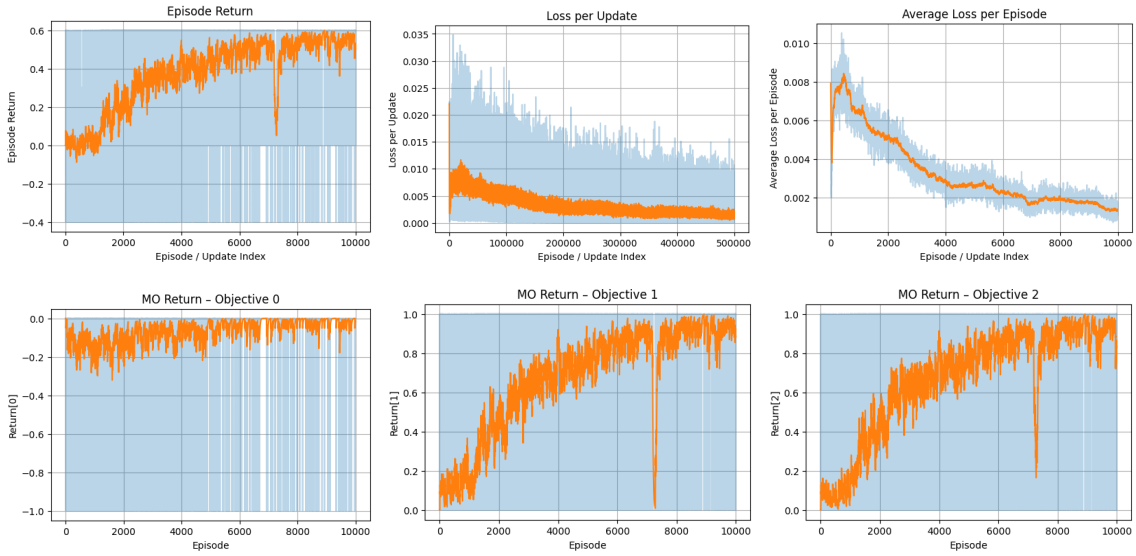

Figure 2: resource-gathering-v0 training results.

Figure 3 shows training results for mo-mountaincar-v0 environment. For a constant number of timesteps, the number of forward/backward actions required to reach the top of the mountain reduces across episodes, with the episode return converging to -40. The average loss reduces significantly early in the updates, with lower reduction rate for the rest of the updates.

Figure 4 shows training results for deep-sea-treasure-v0 environment. Results converge to positive rewards across episodes, with subobjective 0 (treasure value found) converging to 15, and subobjective 1 (time penalty) converging to -10. A balance in the subobjectives is achieved, which can be modified at inference. Similar to mountaincar environment, convergence is achieved early in the updates.

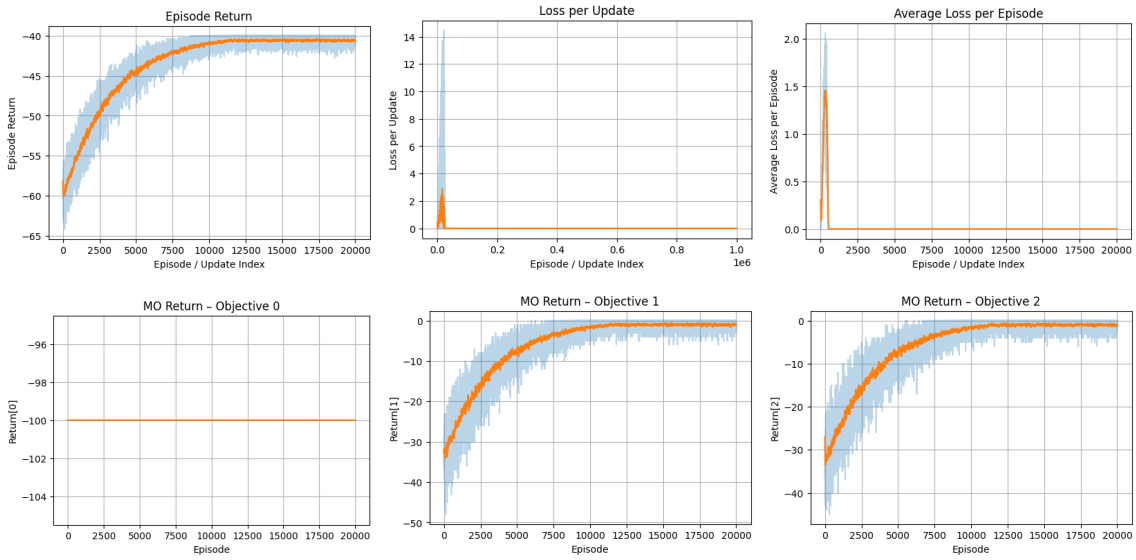

Figure 3: mo-mountaincar-v0 training results.

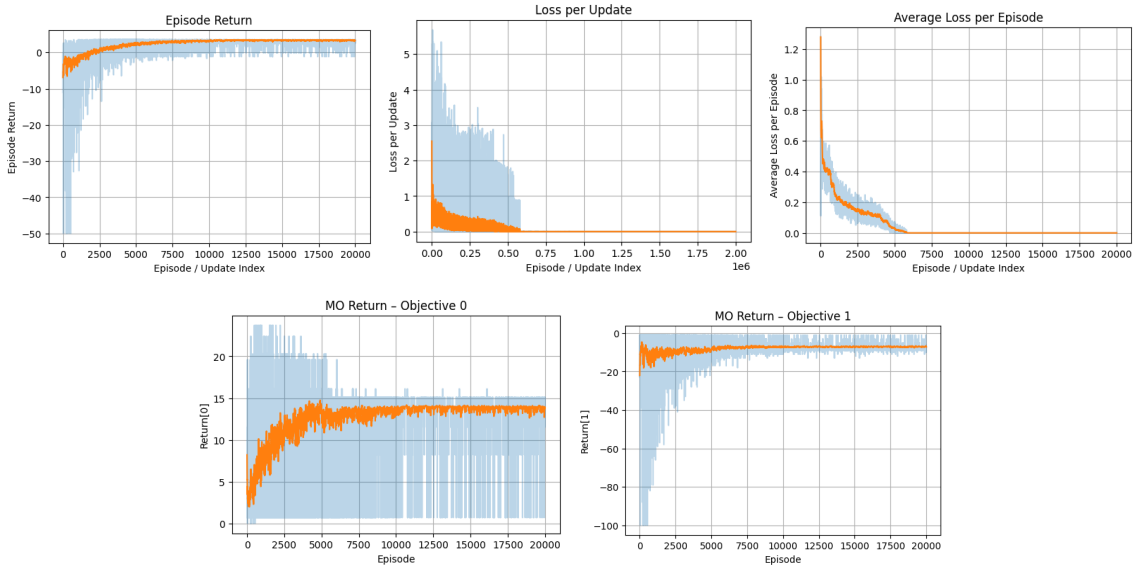

Figure 4: deep-sea-treasure-v0 training results.

4.2 Performance of MODQN vs MONES in water-reservoir-v0

We trained MODQN and MONES algorithm in the water-reservoir-v0 environment. This environment has a continuous state-action space with 2-dimensional reward (flooding and water demand). The utility function for MODQN linearly combines the weighted subobjectives. Table 3 shows the fine-tuned hyperparameters for the MODQN algorithm, and figure 5 shows a plot of the expected scalarized returns across varying weights for flooding. The linear dependence of expected returns on weights suggest impact of utility function during inference, with varying importance to subobjectives.

The expected return of MONES algorithm in water-reservoir-v0 environment is shown in Figure 6. The impact of subobjective weight on reward is minimal for MONES. Hence, MODQN algorithm proposed is more reactive to the utility function used at inference. However, the scalarized reward achieved by MONES is better, due to 30 evolutionary steps, each optimizing the non-dominance ranking and clustering objectives.

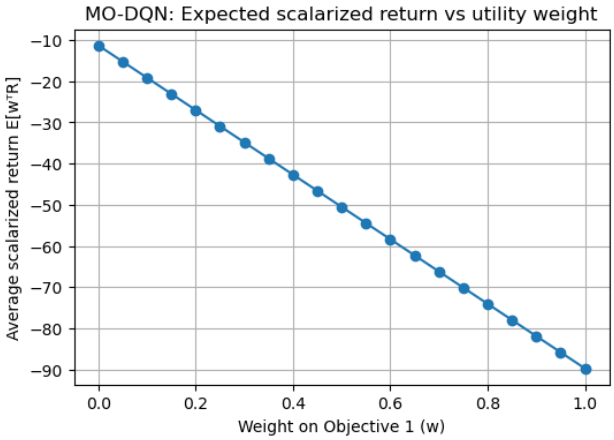

Figure 5: Expected scalarized return of MODQN algorithm in water-reservoir-v0 environment

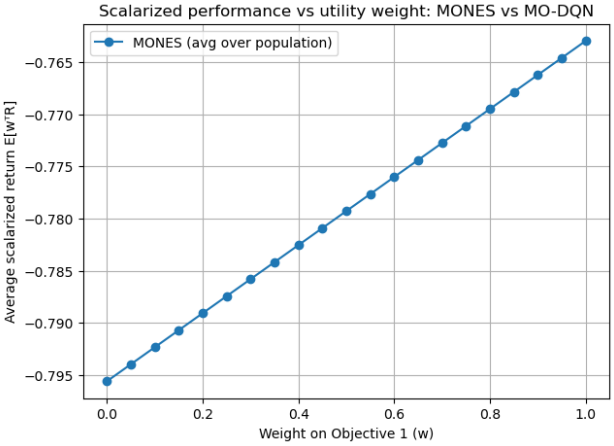

Figure 6: Expected scalarized return of MONES algorithm in water-reservoir-v0 environment

5 Discussion

In this paper, we have proposed MODQN, a single-policy linear DQN with a multi-objective reward vector. We compare the performance with multi-policy evolutionary algorithm MONES, and report scalability of MODQN across a diverse range of environments. Our algorithm achieves high rewards across many sub-objectives in 4 multi-objective RL environments, which SoTA algorithms struggle to achieve (Nguyen et al. [2020]). The algorithm also achieves fast convergence in all environments. Moreover, estimated reward with our algorithm also has a strong dependence on the utility function used (higher than MONES), allowing for optimizing specific subobjectives during inference.

We also found that the off-policy updates (with a replay buffer) were sufficient to achieve fast convergence, without necessitating the use of parallel policy learning.

However, as observed in Mossalam et al. [2016] and Hayes et al. [2021], multi-policy learning can be explored to further improve rewards, especially in continuous action-space environments.

6 Future Work

As a long-term research direction—beyond the scope of this course project—we also aim to investigate the following fundamental question:

Given a utility function, can inverse reinforcement learning be employed to reconstruct the corresponding reward vector?

The primary motivation for this inquiry is to align human-interpretable utilities with machine-learnable rewards. Such a mapping would enable the extraction of individual objectives for separate optimization, enhance interpretability, and facilitate transfer across domains. A persistent challenge in reinforcement learning—highlighted notably in RLHF (Reinforcement Learning from Human Feedback) [Ouyang et al., 2022]—is translating qualitative user preferences into quantitative reward signals. Developing a principled method to reconstruct reward functions from utility specifications would represent a significant step toward improving the modularity, transferability, interpretability, and robustness of multi-objective decision-making systems.

References

Felten, F., Alegre, L. N., Nowé, A., Bazzan, A. L. C., Talbi, E. G., Danoy, G., and Silva, B. C. da. (2023). A toolkit for reliable benchmarking and research in multi-objective reinforcement learning. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*.

Hayes, C. F., Rădulescu, R., Bargiacchi, E., Källström, J., Macfarlane, M., Reymond, M., Verstraeten, T., Zintgraf, L. M., Dazeley, R., Heintz, F., et al. (2021). A practical guide to multi-objective reinforcement learning and planning. *arXiv preprint arXiv:2103.09568*.

Mossalam, H., Assael, Y. M., Roijers, D. M., and Whiteson, S. (2016). Multi-objective deep reinforcement learning. *arXiv preprint arXiv:1610.02707*.

Nguyen, T. T., Nguyen, N. D., Vamplew, P., Nahavandi, S., Dazeley, R., and Lim, C. P. (2020). A multi-objective deep reinforcement learning framework. *Engineering Applications of Artificial Intelligence*, 96:103915.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.